

Fundamentos de Ponteiros em Linguagem C

// Atividade de Consolidação e Aprofundamento — Semana 13

ORIENTAÇÕES DE ENTREGA

DATA DE ENTREGA

21 / 05 / 2026

LOCAL DE ENTREGA



GitHub

FORMATO

.c (código-fonte)

PASTA NO REPOSITÓRIO

PEM-20261-Semana13/

01 Conceito e Definição de Ponteiros

Imagine que cada variável do seu programa ocupa um endereço físico na memória RAM, como uma casa em uma rua. Um ponteiro não guarda um valor qualquer — ele guarda o endereço dessa casa. Dominar ponteiros é o que separa quem "usa" C de quem verdadeiramente programa em C: eles são a base de alocação dinâmica, estruturas de dados avançadas e comunicação direta com o hardware.

DEFINIÇÃO

Um **ponteiro** é uma variável que armazena o endereço de memória de outra variável. Declarado com o operador `*` antes do nome, seu tipo deve corresponder ao tipo da variável apontada.

```
int valor = 42;
int *ptr;      /* declara ponteiro para int */
ptr = &valor;  /* ptr recebe o endereço de valor */
```

C

▶ EXERCÍCIOS

Exercício 01

Endereço na Memória

Declare variáveis dos tipos `int`, `float` e `char`, atribua valores a elas e crie um ponteiro para cada uma. Imprima na tela, para cada variável:

- O valor da variável;

- O endereço de memória da variável (usando `%p`);
- O endereço armazenado no ponteiro;
- O valor obtido a partir do ponteiro.

Exercício 02

Tamanho de um Pontoeiro

Utilizando o operador `sizeof`, imprima o tamanho (em bytes) de ponteiros para `int`, `float`, `double` e `char`. Comente no código: o tamanho varia com o tipo apontado? Por quê? Inclua a explicação como comentário em C (`/* ... */`).

02 Operador de Referência (&)

Se ponteiros são o mapa, o operador & é a bússola: ele aponta para onde uma variável vive na memória. Sem ele, não há como conectar um ponteiro ao dado que ele deve representar. Entender quando e por que usar & é o primeiro passo para escrever código C seguro e eficiente.

DEFINIÇÃO

O operador **de referência** (&) retorna o endereço de memória de uma variável. É usado para inicializar ponteiros ou para passar o endereço de uma variável a uma função.

```
int x = 10;
int *p = &x;    /* p recebe o endereço de x */
printf("Endereço: %p\n", (void*)p);
```

C

► EXERCÍCIOS

Exercício 03

Troca de Valores via Referência

Implemente uma função `trocar(int *a, int *b)` que troca o conteúdo de duas variáveis inteiras usando o operador & na chamada. Demonstre, no `main`, que os valores foram efetivamente trocados, imprimindo antes e depois da chamada.

Exercício 04

Mapa de Endereços de um Struct

Declare um struct `Aluno` com campos `nome` (`char[50]`), `matricula` (`int`) e `media` (`float`). Crie uma instância,

popule-a e imprima o endereço de cada campo individualmente usando `&`. Observe e explique (em comentário) por que os endereços são consecutivos ou não.

03 Operador de **Desreferenciação** (*)

Ter o endereço de uma variável é ter o poder de acessá-la e modificá-la de qualquer ponto do programa. O operador `*` — quando aplicado a um ponteiro — funciona como uma chave: abre a memória no endereço guardado e revela (ou altera) o valor lá armazenado. É o mecanismo central de toda a manipulação indireta de dados em C.

DEFINIÇÃO

O operador **de desreferenciação** (*), aplicado a um ponteiro, acessa o valor armazenado no endereço que o ponteiro contém. Pode ser usado tanto para leitura quanto para escrita.

```
int val = 7;
int *ptr = &val;
*ptr = 99;          /* modifica val indiretamente */
printf("%d\n", val); /* imprime 99 */
```

c

► EXERCÍCIOS

Exercício 05

Modificação Indireta

Declare um vetor de 5 inteiros inicializado com valores quaisquer. Crie um ponteiro para o vetor e, usando **apenas** o operador `*` (sem índices `[]`), multiplique cada elemento por 2. Imprima o vetor antes e depois da operação.

Exercício 06

Leitura e Escrita de Struct via Ponteiro

Crie um struct `Produto` com nome (`char[40]`), preço (`float`) e estoque (`int`). Declare uma instância e um ponteiro para ela. Usando **apenas** desreferenciação com `*` e o operador `.` (sem usar `->`), preencha e exiba todos os campos. Em seguida, repita usando `->` e compare o código produzido.

04 Aritmética de **Ponteiros**

Ponteiros não são números comuns — sua aritmética é calibrada pelo tamanho do tipo apontado. Incrementar um ponteiro `int*` avança exatamente 4 bytes na memória (em arquiteturas de 32/64 bits), não 1. Esse comportamento é a base do acesso eficiente a arrays e é o que torna o C tão próximo do hardware: você controla cada byte.

DEFINIÇÃO

Operações aritméticas em ponteiros (+, -, ++, --) avançam ou recuam o ponteiro em múltiplos do tamanho do tipo apontado. A subtração entre dois ponteiros do mesmo tipo retorna o número de elementos entre eles.

```
int v[] = {10, 20, 30, 40};  
int *p = v;          /* aponta para v[0] */  
p++;                 /* agora aponta para v[1] */  
printf("%d\n", *p); /* imprime 20 */
```

c

▶ EXERCÍCIOS

Exercício 07

Percurso com Incremento

Declare um array de 8 inteiros com valores de sua escolha. Utilizando *apenas aritmética de ponteiros* (sem o operador `[]`), percorra o array e calcule e imprima a soma, a média, o maior e o menor valor. Imprima também o endereço de cada elemento durante o percurso.

Exercício 08

Distância entre Ponteiros

Dado um array de `double` com 10 elementos, inicialize dois ponteiros: um apontando para o 3.º elemento e outro para o 8.º. Calcule a diferença entre os ponteiros e explique (em comentário) como o compilador realiza esse cálculo. Em seguida, percorra o intervalo entre os dois ponteiros imprimindo cada elemento.

05 Ponteiros com Arrays

Em C, o nome de um array é, na prática, um ponteiro constante para seu primeiro elemento. Essa equivalência não é coincidência — é um design deliberado da linguagem que permite manipular coleções de dados com a mesma eficiência de um ponteiro simples. Compreender essa dualidade é essencial para trabalhar com strings, matrizes e buffers de dados em geral.

DEFINIÇÃO

O identificador de um array decai para um ponteiro para seu primeiro elemento. As notações `arr[i]` e `*(arr + i)` são equivalentes. Ponteiros oferecem forma alternativa (e frequentemente mais eficiente) de iterar sobre arrays.

```
int arr[] = {5, 10, 15};  
int *p    = arr;      /* equivalente a &arr[0] */  
printf("%d\n", arr[1]); /* 10 */  
printf("%d\n", *(p+1)); /* 10 — equivalente */
```

c

▶ EXERCÍCIOS

Exercício 09

Inversão de Array com Ponteiros

Escreva uma função `inverter(int *arr, int n)` que inverta, *in-place*, um array de inteiros de tamanho `n` usando dois ponteiros — um iniciando no começo e outro no fim — que se aproximam até cruzar. Demonstre chamando a função com um array de 7 elementos e imprimindo antes e depois.

Exercício 10

Busca em Matriz com Ponteiro

Declare uma matriz 4×4 de inteiros. Utilizando um único ponteiro para `int` que percorre a matriz como se fosse um array linear (explorando a disposição contígua em memória), encontre e imprima a posição (linha, coluna) do maior valor da matriz. Não utilize notação `[i][j]` no laço de busca.

06 Passagem de Ponteiros em Funções

Em C, argumentos são passados por valor — a função recebe uma cópia. Mas quando o argumento é um ponteiro, a cópia contém o endereço do dado original: a função passa a ter acesso direto à memória do chamador. Esse mecanismo, conhecido como *passagem por referência simulada*, é a chave para funções que modificam variáveis externas, retornam múltiplos resultados e manipulam grandes estruturas sem custo de cópia.

DEFINIÇÃO

Ao passar um ponteiro como parâmetro, a função recebe o endereço da variável original e pode modificar seu conteúdo via desreferenciação. Isso permite "retornar" múltiplos valores e modificar variáveis do escopo chamador.

```
void dobrar(int *n) {  
    *n *= 2;      /* modifica a variável original */  
}  
  
int main() {  
    int x = 5;  
    dobrar(&x);  
    printf("%d\n", x); /* imprime 10 */  
    return 0;  
}
```

c

► EXERCÍCIOS

Exercício 11

Função com Múltiplos Retornos

Implemente a função `estatisticas(int *v, int n, int *soma, float *media, int *maior, int *menor)` que, a partir de um array, calcula e armazena nos ponteiros fornecidos a soma total, a média, o maior e o menor valor. Chame a função no `main` com um array de 6 elementos e exiba todos os resultados.

Exercício 12

Ordenação por Seleção com Ponteiros

Implemente o algoritmo *Selection Sort* na função `selectionSort(int *arr, int n)` usando exclusivamente aritmética de ponteiros (sem []). Utilize uma função auxiliar `trocar(int *a, int *b)` para as trocas. Demonstre ordenando um array de 8 inteiros em ordem crescente, exibindo o estado do array antes, durante (a cada troca) e após a ordenação.